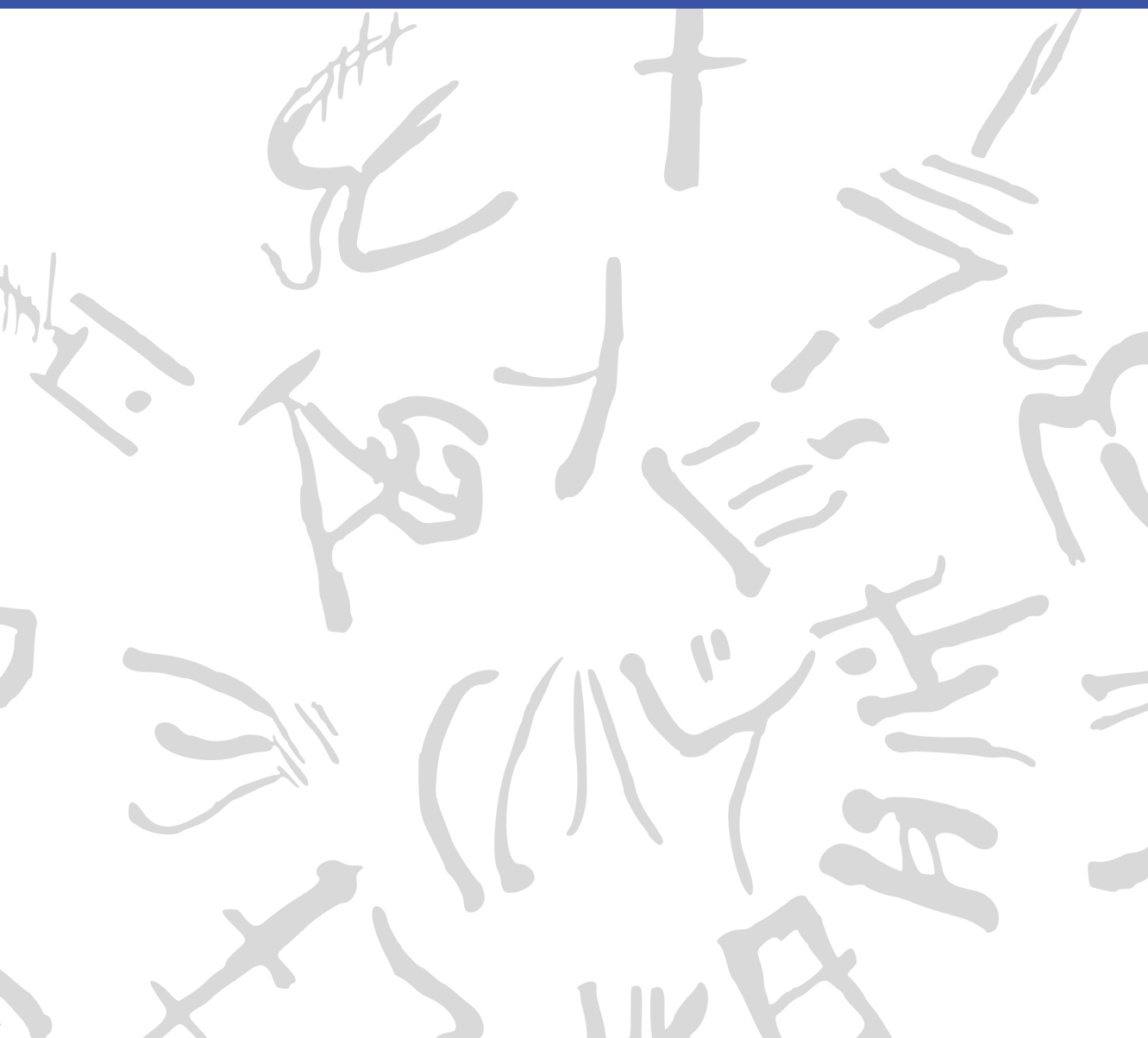


Machine Learning for Semi-Automated Annotations of the Linear A Inscriptions

Bachelor Thesis



DTU Compute

Department of Applied Mathematics and Computer Science
Technical University of Denmark

Richard Petersens Plads

Building 324

2800 Kongens Lyngby, Denmark

www.compute.dtu.dk

Preface

This Bachelor's thesis was prepared at the Department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a Bachelor of Science in Engineering (BSc Eng) degree in General Engineering (Cyber Systems).

Kongens Lyngby, April 15, 2026

A handwritten signature in black ink, enclosed within a roughly drawn, rounded rectangular border. The signature appears to be 'F. W. L. Christoffersen' written in a stylized, cursive-like font.

Frederik W. L. Christoffersen

Abstract

This thesis is in the workings... [\[Write something when finished with all thesis chapters\]](#)

Acknowledgements

Frederik W. L. Christoffersen

General Engineering (Cyber Systems), DTU

Author of this thesis

Morten Mørup

Professor, DTU Compute

Main supervisor

Ester Salgarella

Research Fellow, AIAS Aarhus / University of Cambridge

Co-supervisor, external collaborator, and creator of SigLA

Lukas Esterle

Associate Professor, Aarhus University

Co-supervisor

Generative AI tools were occasionally used as support for brainstorming, linguistic editing, code exploration, and research during the development of this project.

Contents

Preface	i
Abstract	iii
Acknowledgements	v
Contents	vii
Acronyms	ix
List of Figures	xi
List of Tables	xiii
1 Introduction	1
1.1 Background	1
1.2 Related work	3
1.3 Problem statement	4
1.4 Impact – innovation and application	5
1.5 Overview of the thesis	6
2 Data	7
2.1 Sources	7
2.2 Characteristics and construction	7
2.3 Limitations and biases	9
3 Methodology	11
3.1 Common methodology	11
3.2 Non-interactive segmentation ability	12

3.3	Interactive segmentation ability	30
3.4	Workflow efficiency	30
4	Results and Discussion	33
4.1	Non-interactive segmentation performance	33
4.2	Interactive segmentation performance	40
4.3	Workflow impact	40
5	Conclusion	41
5.1	Summary of findings per research question	41
5.2	Overall contribution	41
5.3	Future Work	41
A	An Appendix	43
A.1	Non-interactive segmentation results	43

Acronyms

RQ	Research Question
ML	Machine Learning
CV	Computer Vision
GAN	Generative Adversarial Network
HT	Haghia Triada Tablet
SAM	Segment Anything Model
mSAM	MobileSAMv2
SA-1B	Segment Anything 1 Billion
ViT	Vision Transformer
CNN	Convolutional Neural Network
NLP	Natural Language Processing
LLM	Large Language Model
MLP	Multilayer Perceptron
Otsu	Otsu's method
Gaussian	Gaussian adaptive thresholding with bilateral pre-filtering
CannyFill	Canny edge detection with morphological closing
GC+brush	GrabCut with brush seeding

mSAM+pts MobileSAMv2 with automatic point prompts

YOLO You Only Look Once

Cefael Collections de l'École française d'Athènes en ligne

GORILA Godart and Olivier, Recueil des inscriptions en linéaire A

PDR Project Definition Report

IPR Intellectual Property Rights

GMM Gaussian Mixture Model

IoU Intersection over Union

Dice Dice-Sørensen coefficient

TPE Tree-structured Parzen Estimator

List of Figures

2.1	Example of a raw inscription image and its corresponding ground truth annotation.	8
2.2	Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.	10
3.1	Hyperparameters for Gaussian adaptive thresholding with bilateral pre-filtering (Gaussian).	16
3.2	Hyperparameters for Canny edge detection with morphological closing (CannyFill).	18
3.3	Hyperparameters for GrabCut with brush seeding (GC+brush).	20
3.4	A visual example of seed distribution for GrabCut with brush seeding (GC+brush) on HT6b. Green overlay represents sure foreground pixels, while red overlay represents sure background pixels, and white overlay represents probable background pixels.	21
3.5	Architecture of SAM, from mobile_sam	22
3.6	Architecture of SAM's mask decoder, from sam	24
3.7	Hyperparameters for MobileSAMv2 with automatic point prompts (mSAM+pts)	26
3.8	A visual example of prompt distribution for MobileSAMv2 with automatic point prompts (mSAM+pts), overlaid on HT6b. Green points represent foreground point prompts, while red points represent background point prompts.	27
4.1	Dice Score performance per SAM variant. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance.	34
4.2	Dice score comparison of all models on the evaluation dataset.	35

4.3	Visual comparison of output masks per model from segmentation on HT6b (medium), shown with Dice scores next to the raw image and ground truth.	36
4.4	QQplot of the residuals of the paired t-test comparing GC+brush and mSAM+pts. The residuals appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test for normality of these residuals yielded a p-value of $0.647 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected.	38

List of Tables

A.1	Best TPE hyperparameter optimization results for CannyFill after 100 trials.	43
A.2	Best TPE hyperparameter optimization results for Gaussian after 100 trials.	43
A.3	Best TPE hyperparameter optimization results for mSAM+pts after 100 trials.	44
A.4	Best TPE hyperparameter optimization results for GC+brush after 100 trials.	44
A.5	Raw Dice score data on easy images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.	45
A.6	Raw Dice score data on medium images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.	46
A.7	Raw Dice score data on hard images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.	46
A.8	Summary Statistics per Model of Dice Score on all images. .	47
A.9	Summary Statistics per Model of Dice Score on easy images.	47
A.10	Summary Statistics per Model of Dice Score on medium images.	47
A.11	Summary Statistics per Model of Dice Score on hard images.	48

CHAPTER 1

Introduction

1.1 Background

When the British archaeologist Arthur Evans in the beginning of the 20th century went to excavate Knossos on Crete, he found a huge amount of clay tablets written in a previously unknown linear script. Some documents were clearly of a newer script, which he called *Linear B*, while others were of an older one, which he named *Linear A*. Some fifty years later, the British amateur archaeologist Michael Ventris finally succeeded in deciphering the newer script Linear B, which he proved to be representing Mycenaean Greek, the earliest attested form of Greek. But the same hypothesis did not hold for the older script Linear A, whose encoded language remained unknown, still yet to be deciphered.

A key obstacle to its decipherment has been the small size of its corpus, lately consisting of only around 2,500 inscriptions (about 1,400 if only including those well-preserved enough to be readable) - significantly smaller than that of its younger counterpart that now spans more than 4,500 inscriptions (which are comparatively longer and better preserved).^{1,2} But new inscriptions are still being discovered, with the corpus most recently extended in 2025 by over 100 new documents.^{3,4}

Another challenge has been the lack of digital representations of these inscriptions, in a form suitable for statistical analysis. While photographs with accompanying drawings of nearly the entire corpus are available online through Collections de l'École française d'Athènes en ligne (Cefael) (see **EtudesCretoises21**), these are only available in print form. This

¹Salgarella2020.

²Salgarella2025.

³RILA-S1.

⁴CNRLinearACorpus2025.

motivated the development of the palæographical⁵ database *The Signs of Linear A* (SigLA), which seeks to "[develop] a systematic, exhaustive and user-friendly open access database of all Linear A inscriptions [...] in order to carry out statistical and palæographic analyses within the epigraphic corpus" (**SalgarellaCastellanSigLA2020**). However, the last addition to SigLA was in 2021, and the project has so far managed to document just 772 of all currently known inscriptions.^{6,7}

1.1.1 A problem of manual digitization

The drawings currently stored on SigLA were made by Ester Salgarella using the painting tool Krita. For each document, she manually created her own annotated drawings based on the originals from Cefael. From these, she proceeded to create layered Krita files with metadata for each sign, which could then at last be imported to SigLA.⁸ According to her (personal communication), this has been a highly time-consuming undertaking. And the necessity of this kind of labor-intensive work has arguably been the biggest reason to why SigLA still does not contain the entire available corpus.

The aim of this project was therefore to investigate whether parts of this process can be automated. More specifically, whether some kind of semi-automatic segmentation tool can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA's database.

1.1.2 A problem of small data

The small size of the Linear A corpus and the irregularities in quality of its documents is a huge problem for the application of deep learning approaches, which typically require large amounts of data to perform well. Consequently, this project instead sought to make use of generalized Machine Learning (ML) and Computer Vision (CV). In

⁵The term "palæographical" refers to the study of ancient writing systems.

⁶**SigLAAboutPage**.

⁷**SigLASEarchDocuments**.

⁸**SalgarellaCastellanSigLA2020**.

particular Meta’s state-of-the-art interactive segmentation model, the Segment Anything Model (SAM) (**sam**), which is pre-trained on a massive and diverse dataset - Segment Anything 1 Billion (SA-1B), of over 1 billion masks. SAM’s generalization and interactivity (taking user-placed prompts that guide the segmentation) potentially allows for semi-automatic annotation of Linear A inscriptions without the need for large amounts of training data, which was unfortunately not available in this context. It was therefore of particular interest to map the capabilities and limitations of SAM within this domain.

1.2 Related work

Within the last decade, the field of document annotation has seen significant advancements through the use of machine learning. Take for example the EU-funded generic CNN-based framework for historical document processing *dhSegment* (**dhsegment**). Or the more recent work on segmentation of stone inscriptions (**DeepLearningSegmentation**), where SAM and other state-of-the-art methods are beaten as benchmarks by their stacked U-Net and GAN architecture. Or even the YOLO and Roboflow based approach to segmentation of Palmyrene Aramaic inscriptions (**YOLOSegmentation**) that augment⁹ their dataset with a Generative Adversarial Network (GAN) to increase performance.

Nevertheless, these methods all depend on deep learning, making them inconvenient for domains of small data, which instead seem to require a generalized and interactive approach such as SAM.

1.2.1 SAM and its applications

Ever since its release in 2023, SAM has been extensively evaluated across a wide range of CV tasks to investigate whether it can really “segment anything”. Papers have evaluated its performance within niche domains, such as medical images (**MedSAM**), camouflaged objects (**CamSAM**), transparent objects (**TransparentSAM**), hierarchical texts (**HiSAM**),

⁹The term *augmenting* refers to the process of increasing the size of a dataset.

and even oracle bone inscriptions (**OBIFloSAM**). Most of these papers found that SAM still tends to struggle with such domains, despite its impressive zero-shot¹⁰ generalization. Meanwhile, some of them also found that this performance can be significantly improved through architectural extensions and domain-specific fine-tuning, which unfortunately requires deep learning and a lot of data. However, fortunately, it has equally been shown that SAM’s performance can significantly improve through effective prompt engineering alone (**promptSAM**), requiring no deep learning at all.

1.3 Problem statement

As previously stated, the aim of this project is to investigate whether parts of the process of creating annotated drawings for SigLA can be automated. More specifically, whether some kind of semi-automatic segmentation tool using interactive and generalized CV - specifically the Segment Anything Model (SAM) - can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA’s database. But what would be the relative segmentation quality of such a tool? How automatic would it be exactly? And by how much could it improve the efficiency of the workflow? To concretize the research, the problem statement was therefore decomposed into three Research Questions (RQs) - each accompanied by their respective operationalization (see the following subsection for details).

1.3.1 Research questions

1. **RQ1 (Segmentation performance):** *How well does a SAM-based segmentation approach perform quantitatively in terms of segmentation accuracy, compared to simple classical image processing baselines when applied to Linear A document images?*

Operationalization: SAM-based masks will be compared to classical image-processing baselines using overlap metrics (e.g. IoU/Dice)

¹⁰The term *zeroshot* refers to model performance on hitherto unseen data.

against the ground truth segmentations derived from SigLA’s Krita files for a minimum of 25 documents.

2. **RQ2 (Degree of automation):** *To which extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs?*

Operationalization: The number of user interactions (e.g. clicks) required to achieve a certain level of segmentation quality (e.g. IoU/Dice) will be measured against ground truth (SigLA’s Krita files) for a minimum of 25 documents.

3. **RQ3 (Workflow efficiency):** *How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions?*

Operationalization: The time it takes Ester Salgarella to annotate manually compared to being assisted by the proposed segmentation tool will be measured for a minimum of 5 documents.

1.4 Impact – innovation and application

The work of this thesis presents itself as a first proof-of-concept (and a mapping of the difficulties involved) in using SAM and modern CV to accelerate the expansion of the SigLA database. Its main contribution is a prototype of a semi-automatic annotation tool, along with an evaluation framework that can be used to assess the performance of itself and any potential future versions. A future version of this tool could potentially also be applied to Linear B, as it is the whole field of Aegean studies that seems to suffer from severe under-digitization of the primary evidence (which is a huge problem for the scientific investigations of scholars). Hopefully, the work will inspire further research and development within the field of digital epigraphy. Particularly in the application of generalized and interactive CV approaches such as SAM on Linear A, and perhaps even in the broader context of use on small corpus ancient

writing systems, where the lack of big data is ill fit for approaches using deep learning.

1.5 Overview of the thesis

The rest of this thesis is structured into four main chapters (Data, Methodology, Results and Discussion, Conclusion), all split up into sections concerning each of the three research questions.

CHAPTER 2

Data

2.1 Sources

The dataset used in this thesis was constructed from 30 images of raw Linear A inscriptions, and their corresponding ground truth annotations. Cefael (**EtudesCretoises21**) was chosen as the source for raw document images, since it is the only online and widely publicly available source of the printed five volume corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA). And as all raw document images on the pages of this corpus are accompanied by annotated drawings, an argument could be made to make use of those as ground truth annotations. But annotations provided by Ester Salgarella from SigLA (**SalgarellaCastellanSigLA2020**) were chosen instead, as they are more up to date, and are already in a binary mask format that is suitable for comparison with segmentation masks.

What follows in this chapter is a detailed description of this dataset, documenting its characteristics, its construction process, and its limitations and biases.

2.2 Characteristics and construction

The dataset was made to consists of 30 chosen image pairs of raw documents and their corresponding ground truth annotations. No data split was made, as there otherwise would not be enough data for a meaningful evaluation. Thus, the dataset was used both for hyperparameter optimization and final evaluation of the models. Only inscriptions with clay as writing support were included, as they are the most common and representative of the Linear A corpus, and were the only ones for which

SigLA ground truth annotations were already available. The raw document images from Cefael are screenshots of grayscale images within the scanned printed edition of the corpus, and are thus not of the highest quality, being the only widely publicly available images of the Linear A inscriptions. For an example of a raw document image and its corresponding ground truth annotation, see Figure 2.1.

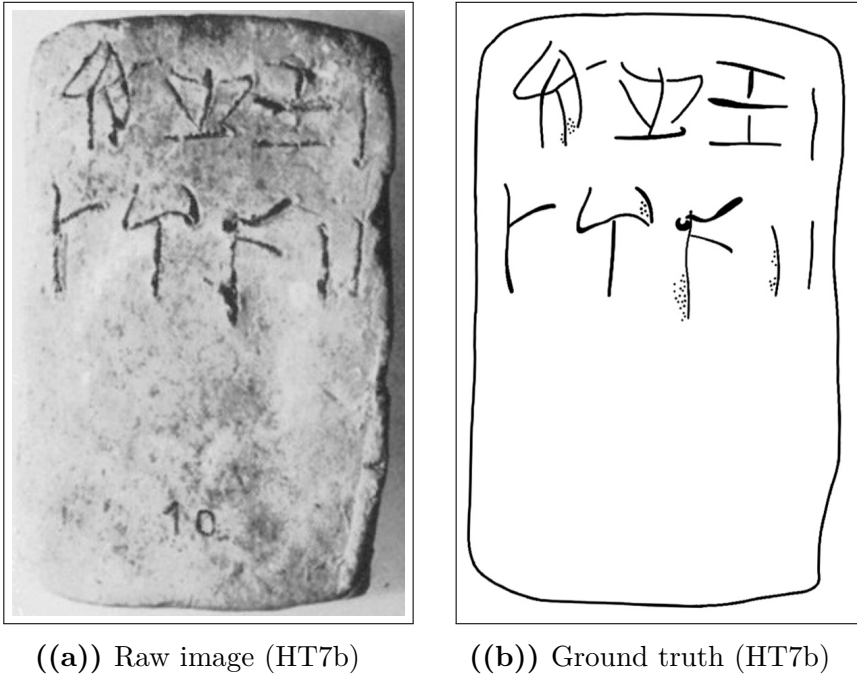


Figure 2.1: Example of a raw inscription image and its corresponding ground truth annotation.

2.2.1 Difficulty classification

The dataset was categorized into three difficulty levels (easy, medium, hard) based on the visual quality of the raw images and the clarity of the inscriptions. This classification was performed manually by inspecting each image and considering factors such as contrast, noise, and the presence of tablet breakages or general damage to the inscriptions. The concrete criteria used to classify the difficulty sets were as follows:

- **Easy:** labeled as such for documents with clear engravings, good contrast, minimal noise, and constant lighting.
- **Hard:** labeled to those with poor contrast, significant noise, and substantial damage making it hard even for the naked eye to read the inscriptions.
- **Medium:** labeled to all those inbetween the qualities of easy and hard.

For visual examples of images within each of these difficulties, see Figure 2.2.

2.2.2 Image registration and preprocessing

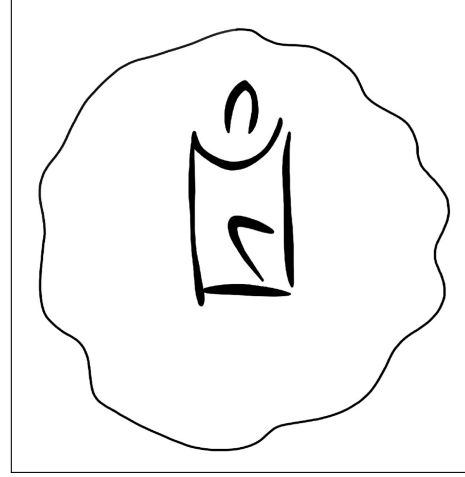
To obtain any kind of meaningful comparison between model outputs and ground truth, the raw images and their ground truth annotations had to be aligned and registered to each other. This was done manually in Krita by downscaling the ground truth images to the individual scales of their raw counterparts, and aligning by translation and rotation, using the raw images as background reference.

2.3 Limitations and biases

Even after image registration, ground truth images still did not perfectly align with the raw images, affecting the obtained segmentation quality. This misalignment is a source of noise in the evaluation.



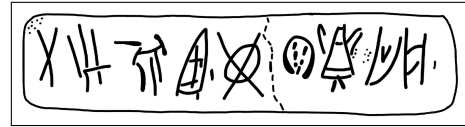
((a)) Easy KHWc2104 (raw)



((b)) Easy KHWc2104 (gt)



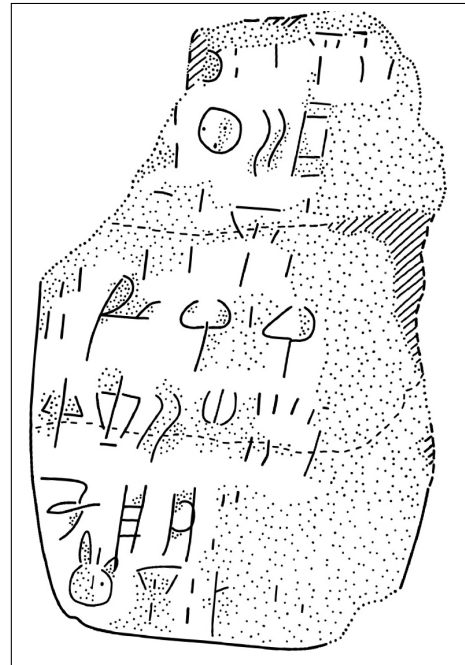
((c)) Medium MA1a (raw)



((d)) Medium MA1a (gt)



((e)) Hard HT3 (raw)



((f)) Hard HT3 (ground truth)

Figure 2.2: Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.

CHAPTER 3

Methodology

3.1 Common methodology

3.1.1 Development environment and tools

Python 3.13 was used for all code development. The choice of this programming language was made due to its big community and library support for nearly everything; be it plotting (Matplotlib), statistics (SciPy), matrix math (NumPy), image processing (OpenCV), or GUI development (PyQt), etc. Python's package manager (pip) also made it exceptionally easy to install and manage dependencies, and set up the development environment. For the ML model implementation library needed (MobileSAMv2), Python was also the language of choice, and thus it was the most convenient for development. The particular version was chosen, as it was the second latest stable release at the time of development, with the risk of Python 3.14 still not having full support by all libraries.

3.1.2 Common theory

[THIS SECTION WILL BE FINISHED LATER, AS IT APPLIES TO ALL RQs. Thinking of ideas for good ways to structure it...]

The basics of spatial filtering can be described by the following equation:

$$g(x, y) = T[f(x, y)]$$

, where $f(x, y)$ (also denoted as $I(x, y)$) is the input image, $g(x, y)$ is the output image.¹

¹**DigitalImageProcessing.**

3.1.2.1 Binary image thresholding

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T \\ 0 & \text{otherwise} \end{cases}$$

2

3.1.2.2 Convolutional kernels

3

$$(w \star I)(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b w(i, j) I(x - i, y - j)$$

where w is the convolution kernel (window) of size $m \times n$, and $a = (m-1)/2$ and $b = (n-1)/2$ are the half-width and half-height (assuming odd dimensions), respectively.⁴

3.2 Non-interactive segmentation ability

To answer the question of RQ1: *How does the initial segmentation quality of SAM compare to that of classic segmentation methods?*, the initial fully-automatic segmentation quality of a computationally efficient implementation of SAM (MobileSAMv2) was evaluated against a set of baseline models, using a suitable segmentation metric (i.e. Dice score) computed from the pairs of predicted and ground truth masks. All models were tuned by hyperparameter optimization to their best performance on the dataset, and the best performing version of SAM was compared against the best performing baseline model.

The following subsections therefore document the configuration of the baseline and state-of-the-art models, as well as the evaluation protocol used to answer RQ1.

²DigitalImageProcessing.

³DigitalImageProcessing.

⁴DigitalImageProcessing.

3.2.1 Baseline model configuration

The baseline models against which the best SAM implementation was evaluated (from simplest to most advanced) were:

- Global thresholding (Otsu’s method)
- Local adaptive thresholding (Gaussian) with bilateral pre-filtering
- Edge-based segmentation (Canny edge detection with region filling)
- Energy-based segmentation (GrabCut with brushseeds)

Together, they cover a wide range of classic segmentation techniques that might be used for binary foreground extraction. What follows is a brief description of each of these methods, including their characteristics, how they work, and their implementation details.

3.2.1.1 Global thresholding (Otsu’s method)

Otsu’s method is a global thresholding method that works on gray-level histograms, which was originally proposed by **Otsu1979**. As the simplest of the baselines, it represents the minimum acceptable segmentation quality, since it only takes the global gray level distribution into account, with no assumptions about any spatial relationships between pixels. It is therefore expected to perform poorly on images with erosion, uneven lighting, and differences in surface texture⁵, which are the exact characteristics of the images to be segmented.

Here is how it works: First, let pixels of the raw image be divided into L shades of gray $[1, 2, \dots, L]$. Then, let each pixel be assigned one of two classes C_0 or C_1 , guided by a threshold T , where C_0 contains the shades $[1, 2, \dots, T]$, and C_1 contains the rest $[T + 1, T + 2, \dots, L]$. Otsu’s method then works by performing an exhaustive search over all L gray levels to find an optimal threshold T^* that results in a maximum between-class variance σ_B^2 :

$$T^* = \arg \max_{1 \leq T < L} \sigma_B^2(T)$$

⁵**DigitalImageProcessing.**

The between-class variance σ_B^2 is then defined by a function of the respective total probabilities of the two classes (ω_0 and ω_1) and the mean gray level value per class (μ_0 and μ_1):

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_1(T) - \mu_0(T)]^2$$

3.2.1.2 Local adaptive thresholding (Gaussian) with bilateral pre-filtering

Local adaptive thresholding works by computing a threshold for each pixel based on its local neighborhood. This is an advantage in cases where the image has varying light conditions (which is the case for the images to be segmented), as the threshold adapts to local lighting.

Mathematically, the local threshold T is a function of a given pixel position (x, y) computed by a weighted sum of the neighborhood. The generated binary mask B is then defined as:⁶

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T(x, y) \\ 0 & \text{otherwise} \end{cases}$$

To compute $T(x, y)$, there are a few weighting functions to choose from. For this baseline, the Gaussian kernel was used, as it can limit the influence of distant noisy pixels by assigning weights based on distance from the kernel center. With a Gaussian kernel G_σ , the threshold $T(x, y)$ is computed as a Gaussian-weighted sum of local intensities subtracted by a constant C . Using convolution ($\star$), this can be expressed as:⁷

$$T(x, y) = (G_\sigma \star I)(x, y) - C$$

In general, the weight at an offset (i, j) from the kernel center can be written as follows (where α is a normalization constant ensuring that the kernel sums to 1, and σ controls the spatial spread of the kernel):⁸

$$G_\sigma(i, j) = \alpha \cdot \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right)$$

⁶`DigitalImageProcessing`.

⁷`opencv__adaptive__threshold`.

⁸`DigitalImageProcessing`.

As a pre-filtering step, a bilateral filter (TomasiManduchi1998) was applied to the input image before performing the Gaussian thresholding. In this way, more noise could be reduced while preserving inscriptions, as this filter is known to be effective at reducing noise while preserving edges. It takes three hyperparameters: the kernel window diameter d , the spatial standard deviation σ_{space} , and the intensity standard deviation σ_{color} . Discretely, its output intensity $I_{\text{bf}}(p)$ at a given pixel position p (where q is the position of one of its neighbors within the kernel window) can be defined as follows:⁹

$$I_{\text{bf}}(p) = \frac{\sum_q I(q) W_{\text{space}}(p, q) W_{\text{color}}(p, q)}{\sum_q W_{\text{space}}(p, q) W_{\text{color}}(p, q)}$$

Here, the spatial weight based on the distance between pixels p and q , and the color range weight based on the intensity difference between p and q are respectively given by:

$$W_{\text{space}}(p, q) = \exp\left(-\frac{\|p - q\|^2}{2\sigma_{\text{space}}^2}\right)$$

$$W_{\text{color}}(p, q) = \exp\left(-\frac{\|I(p) - I(q)\|^2}{2\sigma_{\text{color}}^2}\right)$$

It was decided to use the same σ value for both spatial and color weights, as this simplified the hyperparameter tuning (see Section 3.2.3.2) without significantly compromising the filter’s performance. This gave in total 4 hyperparameters for this baseline (see Figure 3.1 for a list of these):

⁹Pan2025InnovativeBilateral.

X

1. C : the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.2.1.2),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.2.1.2),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral pre-filtering (see Section 3.2.1.2),
4. σ : the spatial and intensity standard deviation for bilateral pre-filtering (see Section 3.2.1.2),

Figure 3.1: Hyperparameters for Gaussian adaptive thresholding with bilateral pre-filtering (Gaussian).

3.2.1.3 Edge-based segmentation (Canny edge detection with region filling)

Canny edge detection is a method that by itself only segments the edges of an image, excluding the enclosed regions within those edges. Here is how it works: First, the input image is smoothed using a Gaussian filter to reduce noise (using the Gaussian kernel as defined in the previous section):

$$I_s(x, y) = (G_\sigma \star I)(x, y)$$

Then, for the image gradient of each pixel, the magnitude $M_s(x, y)$ and the orientation $\theta_s(x, y)$ are computed:¹⁰

$$M_s(x, y) = \|\nabla I_s(x, y)\| = \sqrt{\left(\frac{\partial I_s}{\partial x}\right)^2 + \left(\frac{\partial I_s}{\partial y}\right)^2}$$

$$\theta_s(x, y) = \arctan\left(\frac{\partial I_s / \partial y}{\partial I_s / \partial x}\right)$$

Next, *non-maxima suppression*¹¹ is applied to retain only local maxima of $M_s(x, y)$ along the gradient direction $\theta_s(x, y)$, resulting in thinner edges. At last, this is followed by *hysteresis thresholding*¹², classifying pixels as strong, weak, or non-edges with a low threshold T_l and a high threshold T_h , only preserving weak edges if connected to strong edges.

Inspired by (**autocanny**), these thresholds were automatically determined by setting

$$T_l = \max(0, (1 - \sigma)v), \quad T_h = \min(255, (1 + \sigma)v)$$

turning σ into a hyperparameter that together with the grayscale median of the given image v controlled the sensitivity of the edge detection.

In an attempt to fill the regions enclosed by the detected edges, morphological closing (dilation followed by erosion) was applied to the edge mask. This gave in total 2 hyperparameters (see Figure 3.2 for a list of these).

¹⁰DigitalImageProcessing.

¹¹DigitalImageProcessing.

¹²DigitalImageProcessing.

X

1. σ : the constant used to determine the thresholds T_l and T_h for the hysteresis thresholding,
2. $d_{closing}$: the diameter of the kernel window for the morphological closing of the edges

Figure 3.2: Hyperparameters for Canny edge detection with morphological closing (CannyFill).

3.2.1.4 Energy-based segmentation (GrabCut with brushseeds)

GrabCut (**Rother2004GrabCut**) is a classical interactive segmentation method, which makes for a good comparison to a deep learning interactive model like SAM. It is good at producing spatially coherent masks with little large-grained noise, which is ideal for the images to be segmented.

The algorithm is interactively initialized by the user, who traditionally provide a bounding box around the object of interest. Pixels outside this box are marked as definite background, while pixels inside it are marked as initially unknown. To further guide the segmentation, the user also has the option to provide an additional number of pixels as definite foreground ($B_n = 1$), and definite background ($B_n = 0$).

As an energy-based segmentation method, optimal segmentation is achieved by minimizing a cost (energy) function E that combines a data term D and a smoothness term S , given an input image I , a binary segmentation mask B , and a set of parameters θ :¹³

$$E(B, I, \theta) = D(B, I, \theta) + S(B, I)$$

The set of parameters θ then define two *Gaussian Mixture Models* (GMMs) (probabilistic models each consisting of a weighted sum of several Gaussian distributions) that over iterations will be tuned to model the foreground and background pixel distributions. These are

¹³This is a simplified version of the energy function that is used by GrabCut.

used to compute the data term D whose energy contribution decreases the better B matches the GMMs of foreground and background. The smoothness term S then penalizes neighboring pixels that have different labels, encouraging spatial coherence in the segmentation.

At last, the optimal segmentation B^* is found iteratively, by alternating between optimizing the GMM parameters θ to model B , and minimizing the energy function with respect to B :

$$B^* = \arg \min_B E(B, I, \theta)$$

The automatic brush seeds given to GrabCut (see Figure 3.4 for a visual example) were derived from a Gaussian thresholded mask with bilateral pre-filtering (see Section 3.2.1.2). The foreground of the mask came to represent sure foreground pixels, while an eroded version of the background (the inverse of the mask) represented sure background pixels. A probable background region was then filled in taking up the remaining space created from the eroded background. This gave in total 6 hyperparameters (see Figure 3.3 for a list of these).

X

1. $d_{bilateral}$: the diameter of the kernel window for bilateral pre-filtering (see Section 3.2.1.2),
2. σ : the spatial and intensity standard deviation for bilateral pre-filtering (see Section 3.2.1.2)
3. C : the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.2.1.2),
4. $d_{gaussian}$: the diameter of the kernel window for the Gaussian adaptive thresholding (see Section 3.2.1.2),
5. $d_{prb_erosion}$: the diameter of the kernel window for the erosion of the sure background, filled in by probable background,
6. $iters$: the number of iterations for GrabCut to run.

Figure 3.3: Hyperparameters for GrabCut with brush seeding (GC+brush).

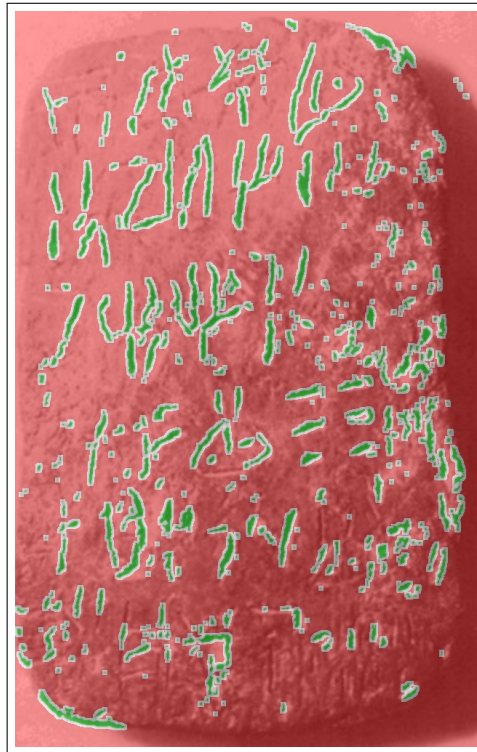


Figure 3.4: A visual example of seed distribution for GrabCut with brush seeding (GC+brush) on HT6b. Green overlay represents sure foreground pixels, while red overlay represents sure background pixels, and white overlay represents probable background pixels.

3.2.2 State-of-the-art model configuration

To be compared against the classical baselines was the generalized deep learning Segment Anything Model (SAM) (**sam**). For the sake of computational efficiency, a mobile implementation of SAM - MobileSAMv2 (mSAM) (**mobile_sam_v2**) was used instead of the original. It is a lightweight version, which was designed to run more efficiently on mobile devices with a similar segmentation capability. More specifically, it is trained on a smaller sample of the same dataset of natural images as the original SAM; SA-1B, and thus has roughly the same segmentation capabilities, although with a slightly lower performance.

As gray-scale images of clay tablets with Linear A inscriptions were not part of that dataset, mSAM had to be adapted to the specific characteristics of the domain. But as it could not be further tuned by training on the domain dataset (due to its small size), the only way to do so, was through the use of prompts. Specifically, automatic prompts, to make the non-interactive comparison to the baselines fair.

The main pipeline of mSAM thus ended up consisting of segmentation of the raw image by MobileSAMv2 into a binary mask, preceded by automatic prompt generation. What follows is therefore a brief description of the model architecture, and the automatic prompt generation method used to adapt it to the domain.

3.2.2.1 Model architecture

For both SAM and MobileSAMv2, the model architecture consists of an image encoder, and a prompt-guided mask decoder (see Figure 3.5).

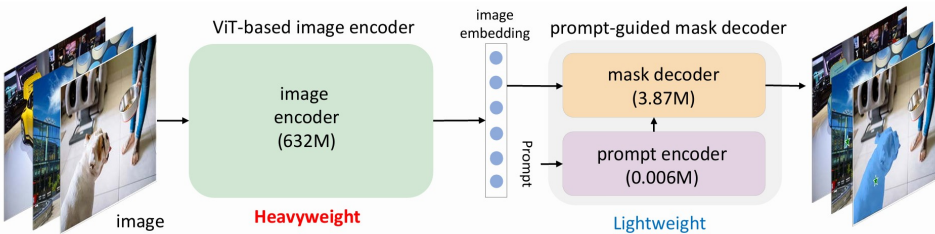


Figure 3.5: Architecture of SAM, from **mobile_sam**.

The image encoder used is a Vision Transformer (ViT)¹⁴ whose purpose is similar to that of a Convolutional Neural Network (CNN), processing an input image to extract its features. But instead of convolutional layers, a ViT makes use of a transformer-based encoder traditionally used in Natural Language Processing (NLP) and Large Language Models (LLMs). As an alternative to words, it treats an image as a sequence of fixed size patches (regions of the image) embedded into vector representations with positional encodings, similar to how tokens work in NLP. These embedded patches are then processed through a repeated series of normalization, feed-forward Multilayer Perceptrons (MLPs), and in-between those, a mechanism known as self-attention that weights the patches according to their relevance to each other.¹⁵

More specifically, given the input embedding X , self-attention computes the projections; queries: Q , keys: K , and values: V , as products of X with respective weight matrices, and passes the information on as a new matrix (where d_k is the dimension of Q and K):

$$Attention(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Here, the softmax function normalizes the output to a probability distribution (putting each element on the interval $]0, 1[$, with their sum being 1), which is then multiplied by V (again turned into logits / non-normalized probabilities).

Thanks to this attention mechanism, after a number of these series-layers, the ViT finally outputs an image embedding containing the spatial context of the image.¹⁶ It is this image embedding, combined with embedded prompts from the prompt encoder (referred to as prompt tokens) that are fed to the mask decoder to generate the final segmentation mask. However, before this happens, the prompt encoder takes any given sparse prompts (i.e. box or point prompts) as embeddings by summing their positional encodings with learned type-specific embeddings

¹⁴The reason that MobileSAMv2 is computationally faster than the original is its lighter ViT implementation TinyViT (**TinyViT** with only 5M parameters) compared to those of SAM (ViT-H, ViT-L, ViT-B with 632M, 307M, and 86M, respectively).

¹⁵**vaswani2023attentionneed.**

¹⁶**imageworth16x16words.**

(one foreground/background embedding per point prompt, and two opposite corner embeddings per box prompt).

These are then fed to the lightweight mask decoder (which is also transformer-based) along with the image-embedding from the ViT, and 3 learnable output token embeddings (see Figure 3.6). Self-attention is applied to the prompt and output tokens, and two-way cross-attention (same as self-attention but with one embedding as queries, and the other as keys and values, and vice-versa) is applied between the tokens and image embedding for them to influence and attend one another. To keep track of spatial coordinates and prompts, positional encodings and the original prompt embeddings are added as input at every attention layer.

At the end of this process, the image embedding is upsampled by convolution to the original image resolution. The three output tokens are then processed in parallel by two separate MLPs: one that generates their final segmentation masks by a dot product with the output and the upscaled image embedding, and another one that outputs a predicted IoU score for each mask. For single output masks, the mask is generated from a fourth output token.¹⁷

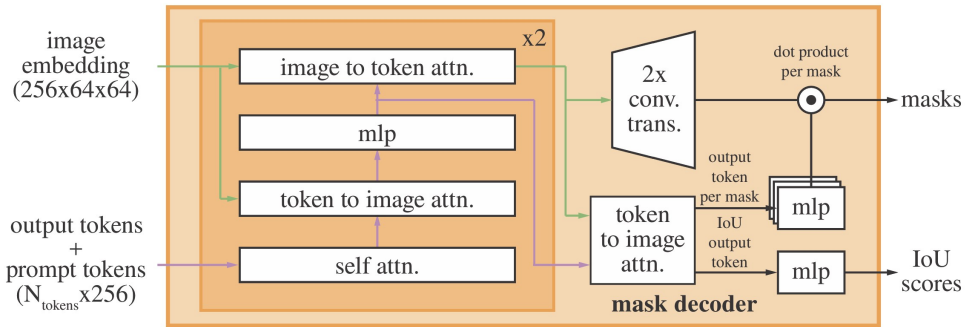


Figure 3.6: Architecture of SAM’s mask decoder, from **sam**.

¹⁷**sam**.

3.2.2.2 Automatic prompting

Even though the MobileSAMv2 implementation already makes use of an automatic box prompt, it does so from a You Only Look Once (YOLO)¹⁸ model trained on a subset of the same dataset as SAM, SA-1B.¹⁹ Those box prompts are generalized for and work best on natural images. Thus, mSAM needed to be adapted through the use of other automatic prompts. But as it was particularly difficult to automatically create high quality box prompts without a trained object detection model, only point prompts were used for this adaptation. To perform a single ablation check of these automatic point prompts, two versions of MobileSAMv2 were then made for comparison:

- **mSAM**: MobileSAMv2 (with the original automatic box prompts from the YOLOv8 model),
- and **mSAM+pts**: MobileSAMv2 with automatic point prompts labeled as either foreground or background.

Just like in the case of GC+brush, the automatic point prompts for mSAM+pts (see Figure 3.8 for a visual example) were derived from a thresholded mask of the Gaussian adaptive thresholding baseline (see Section 3.2.1.2). This was done by randomly sampling a number of foreground and background pixels from the mask as prompts for mSAM+pts. Concretely, the foreground cloud of prompts were sampled from the foreground of the mask by n_{fgd} points, while the background cloud of prompts were sampled from the erosion of the inverse of the mask by n_{bgd} points. This gave in total 7 hyperparameters (see Figure 3.7) for mSAM+pts.

¹⁸YOLO.

¹⁹mobile_sam_v2.

X

1. C : the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.2.1.2),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.2.1.2),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral pre-filtering (see Section 3.2.1.2),
4. σ : the spatial and intensity standard deviation for bilateral pre-filtering (see Section 3.2.1.2),
5. n_{fgd_points} : the number of foreground point prompts to be sampled from the foreground of the first Gaussian thresholded mask,
6. n_{bgd_points} : the number of background point prompts to be sampled from the sure background region,
7. $d_{gap_erosion}$: the kernel size for erosion used to create a gap between the foreground and background regions.

Figure 3.7: Hyperparameters for MobileSAMv2 with automatic point prompts (mSAM+pts)

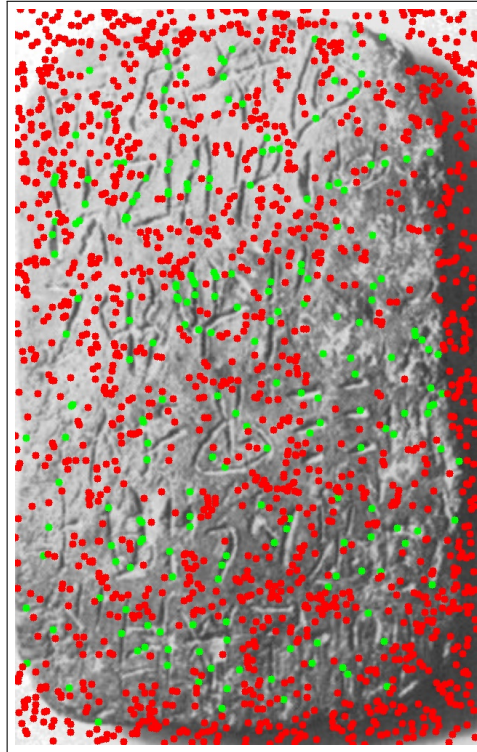


Figure 3.8: A visual example of prompt distribution for MobileSAMv2 with automatic point prompts (mSAM+pts), overlayed on HT6b. Green points represent foreground point prompts, while red points represent background point prompts.

3.2.3 Evaluation protocol and performance metrics

For an evaluation of initial segmentation quality, relevant metrics and statistical tests needed to be defined to determine whether one model performed significantly better than another. Furthermore, all models with hyperparameters needed to be optimized to their best performance on the dataset, to ensure a fair comparison. Naturally, due to the nature of RQ1, it was of particular interest to compare the best performing baseline model against the best performing version of MobileSAMv2.

What follows is therefore a description of the choice of metrics, the optimization of model hyperparameters, and the statistical tests for the evaluation, used to answer RQ1: *How does the initial segmentation quality of SAM compare to that of classic segmentation methods?*

3.2.3.1 Choice of metrics

When it comes to evaluation of segmentation models, the two most commonly used metrics are: 1. the Dice-Sørensen coefficient (Dice) and 2. the Intersection over Union (IoU). The Dice score is most commonly used in cases where there are heavy class imbalances (e.g. lots of background compared to regions segmented), and when foreground objects are small and thin. It is naturally more forgiving than the related segmentation metric IoU, which instead tends to be rather strict, and is specifically used to evaluate segmentation results where small displacements are critical for results. Therefore, segmentation performance was reported using Dice. Mathematically, it is defined as twice the size of the intersection of the predicted segmentation mask $\hat{Y}$ and the ground truth mask Y , divided by their summed size:²⁰

$$\text{Dice}(\hat{Y}, Y) = \frac{2 \cdot |\hat{Y} \cap Y|}{|\hat{Y}| + |Y|}$$

Thus, the raw metrics ended up consisting of the Dice score for each image, and each model.

²⁰**Metrics2025.**

3.2.3.2 Hyperparameter optimization

To ensure a fair comparison between the models, all models with hyperparameters were tuned to their best performance on the dataset using Optuna, a hyperparameter optimization framework. The optimization itself was performed using the Tree-structured Parzen Estimator (TPE) algorithm (**TPEHyperparameterOptimization2013**)²¹, which is a Bayesian optimization method that models the relationship between hyperparameters and performance to efficiently explore the hyperparameter space and find the optimal ones. Hence, it is more sophisticated than a random search, which just samples different hyperparameter combinations without considering the results of past evaluations.

It works by building two probabilistic models: one for the hyperparameter configurations that have resulted in good performance, and another for those that have resulted in poor performance. Then, it uses these models to select the next hyperparameter configuration to evaluate, by maximizing the expected improvement over the current best performance. This way, it can focus the search on promising regions of the space of possible hyperparameters without having to rely on chance or exhaustive search.

3.2.3.3 Statistical tests

To determine whether the observed differences in scores between models were statistically significant, paired t-tests were conducted. The paired t-test has two main assumptions: 1. independence of observations, and 2. normality of residuals. The first assumption is already satisfied a priori, since the segmentation performance of one image does not directly influence the performance on another. However, the second assumption depends on results. Thus, the residuals of the paired differences in Dice scores between the two models in question needed to be checked for normality using a QQ-plot and the Shapiro-Wilk test. And in case the normality assumption was violated, the non-parametric domain-specific Wilcoxon signed-rank test would be used instead.²²

²¹**TPEHyperparameterOptimization2011.**

²²**Rainio2024.**

3.3 Interactive segmentation ability

Here, the interactivity and quality of human-guided SAM (and GrabCut for comparison?) will be evaluated against the initial segmentation results from Section 3.2. [Add some context of RQ2 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

3.3.1 Interactive tool architecture

3.3.2 Experimental design for automation evaluation

3.3.3 Definition of automation metrics

3.4 Workflow efficiency

[Add some context of RQ3 here, and how the question will be operationalized in bigger picture (though more detail than in introduction).]

3.4.1 Export to Krita

[It is indeed possible via the Krita Python API to export the predicted segmentation masks as Krita layers, which can then be further edited by hand by a user. The question is only if I want to do so via the API (requires Krita installed), or via a Python export script. Krita's file format is open, so both are possible.]

3.4.2 Definition of workflow efficiency metrics

3.4.3 Experimental design of the user study

3.4.4 Analysis of results

CHAPTER 4

Results and Discussion

4.1 Non-interactive segmentation performance

After running the non-interactive segmentation evaluation of the models on the dataset (hyperparameter optimization results can be found in Section A.1.1), it was clear that mSAM did not perform better than the classical baselines (see Figure 4.2 for a plot of the results).

Meanwhile, mSAM+pts with the engineered point prompt clouds performed considerably better. With a mean Dice score of around 0.173, it performed better than Otsu and CannyFill (with mean Dice scores of 0.092 and 0.134, respectively), and comparably to the best performing classical baselines, Gaussian and GC+brush that each yielded Dice scores of 0.177 and 0.177, respectively.

The following subsections go into a bit more detail on the explanation of these results, the statistical significance of the differences in performance between mSAM and the best classical baseline (see Section A.1.2 in the appendix for the full results table), and a discussion of the fairness of this experiment.

4.1.1 SAM ablation

The results of the ablation study of different SAM variants (see Figure 4.1) showed that the best performing variant was the one with automatic point prompts and bilateral filter, making use of the single output token per image instead of best of three. This was also the one used for the main evaluation.

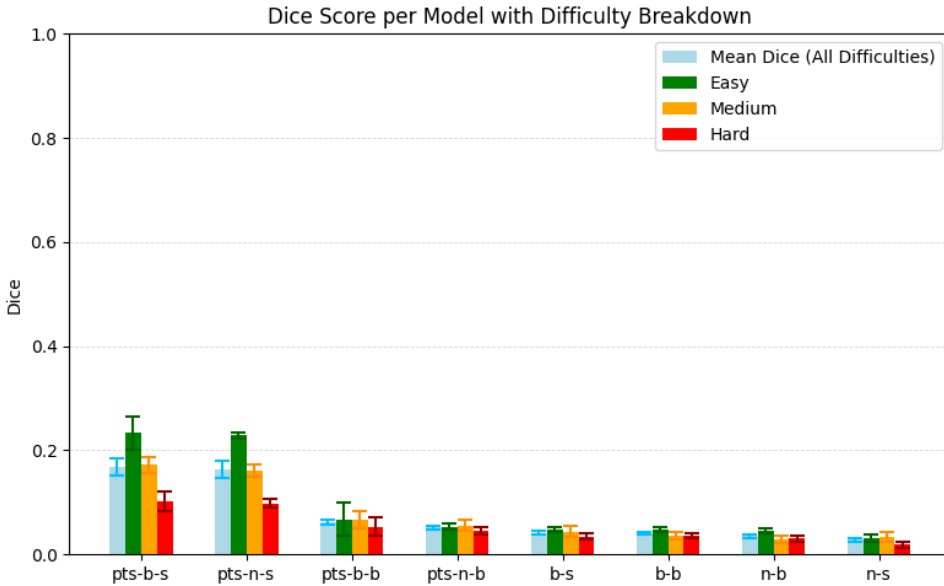


Figure 4.1: Dice Score performance per SAM variant. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance.

4.1.2 Model comparison

Knowing the overall quantitative results (see Figure 4.2), it pays to look at a visual example to see what is going on behind the scenes. In this instance, GC+brush performs decently as well, but oversegments in a way that makes strokes a bit thicker than they should be. It compensates for this, however, by reducing noise. These phenomena are due to a combination of its smoothness term, and its probable background regions, which encourage spatially coherent masks while risking small-scale dilation of the mask (see Section 3.2.1.4).

CannyFill also oversegments, due to its large kernel window size for morphological closing, which was necessary to fill in the gaps between edges, but gives a blocky look, and merges all nearby edges together.

The unprompted mSAM segments only background noise here, producing nothing of value. But the prompted mSAM+pts does produce a somewhat coherent mask, capturing almost every sign with less overseg-

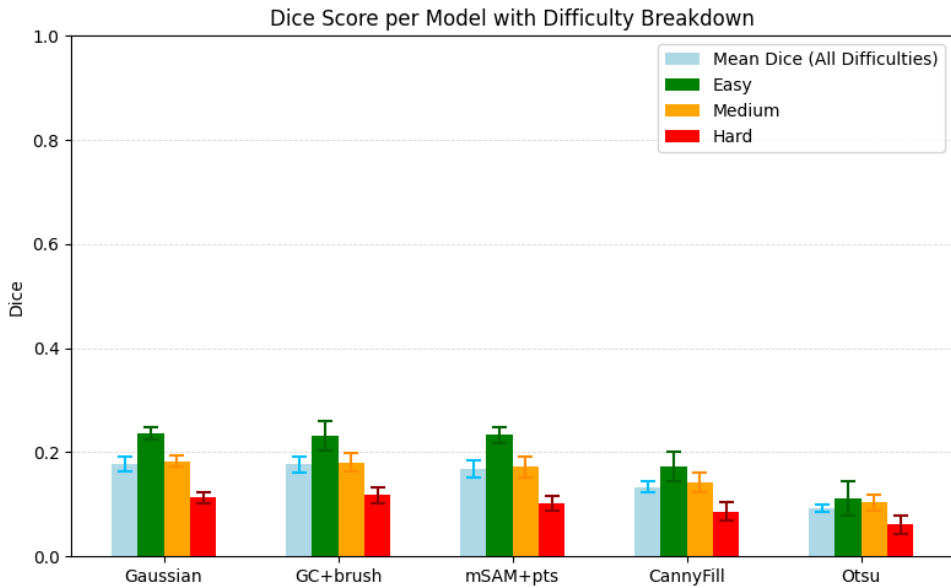
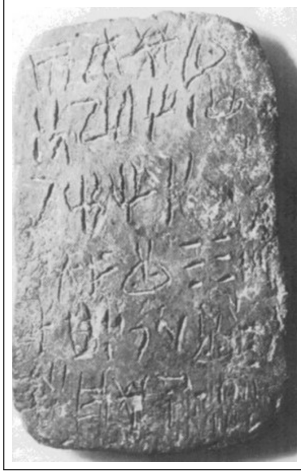
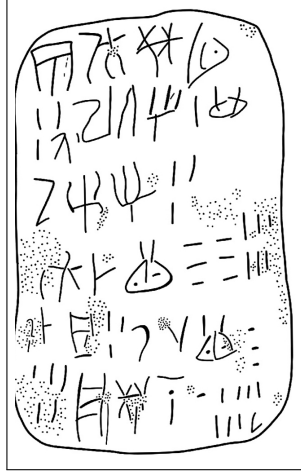


Figure 4.2: Dice score comparison of all models on the evaluation dataset.

mented areas than both Otsu and CannyFill. Still, it oversegments many signs (especially those with background-space supposed to be inside of them), fades out signs where they are supposed to continue (due to loss of attention, as segmenting many separated signs at once is not what its attention weights have been trained for), and is not able to capture the outline of the tablet. But the signs that it does capture well, are captured with a good level of spatial coherence. And there is way less noise between them compared to the masks of the baselines, which is arguably a positive sign for the potential of SAM to perform well on this task once properly fine-tuned prompts, or points are placed through human supervision.



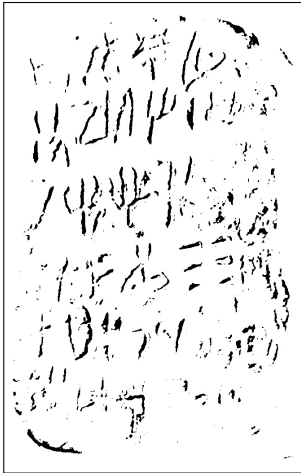
((a)) Raw Image



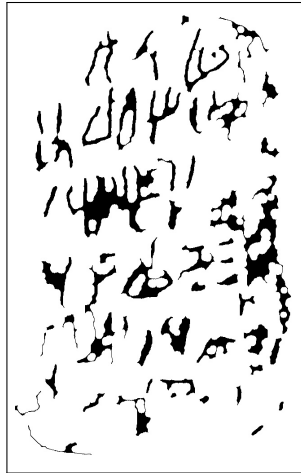
((b)) Ground Truth



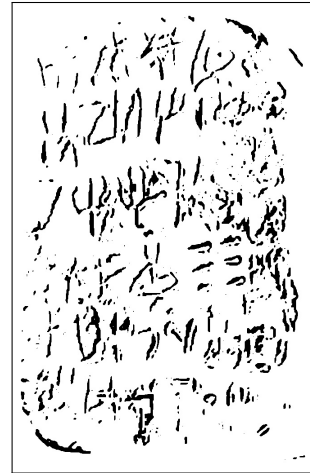
((c)) Otsu 0.118



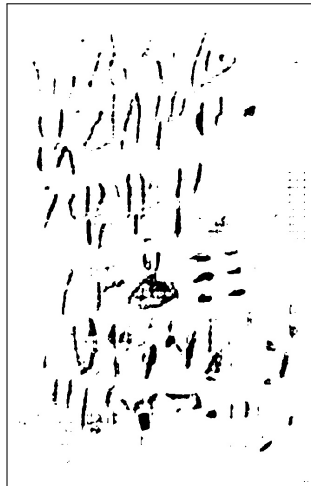
((d)) Gaussian 0.264



((e)) CannyFill 0.227



((f)) GCbrush 0.265



((g)) mSAMpts 0.223

4.1.3 Statistical significance

As it was clear from the results, mSAM+pts did not perform better than the best performing classical baseline. To determine whether it performed significantly worse, a paired t-test was conducted, as residuals of the differences in Dice scores between GC+brush and mSAM+pts were approximately normally distributed (see next Section 4.1.3.1). What follows is the documentation of that normality test, the paired t-test, and the results of both.

4.1.3.1 Normality of residuals

As the QQplot in Figure 4.4 shows, the residuals of the paired differences in Dice scores between GC+brush and mSAM+pts appeared to be approximately normally distributed. Furthermore, the Shapiro-Wilk test for normality of these residuals yielded a p-value of $0.647 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected. Thus, the normality assumption for the paired t-test was satisfied, and its results could be considered valid.

4.1.3.2 Paired t-test

The paired t-test was conducted on the Dice scores obtained for each of the 30 images across GC+brush and mSAM+pts. The null hypothesis (H_0) stated that there is no difference in mean Dice scores between the two models, while the alternative hypothesis (H_1) stated that GC+brush has a higher mean Dice score than mSAM+pts.

Performing the paired t-test yielded a test statistic of $t \approx 9.362$ and a p-value of $P(T > t) \approx 0$. Thus, as $0 < \alpha = 0.05$, the null hypothesis could be rejected, and it could be concluded that GC+brush significantly outperforms mSAM+pts in terms of Dice score. Hence, it can be said that mSAM performed significantly worse in comparison to the best classical segmentation methods for non-interactive segmentation of Linear A tablets, even when automatically prompted.

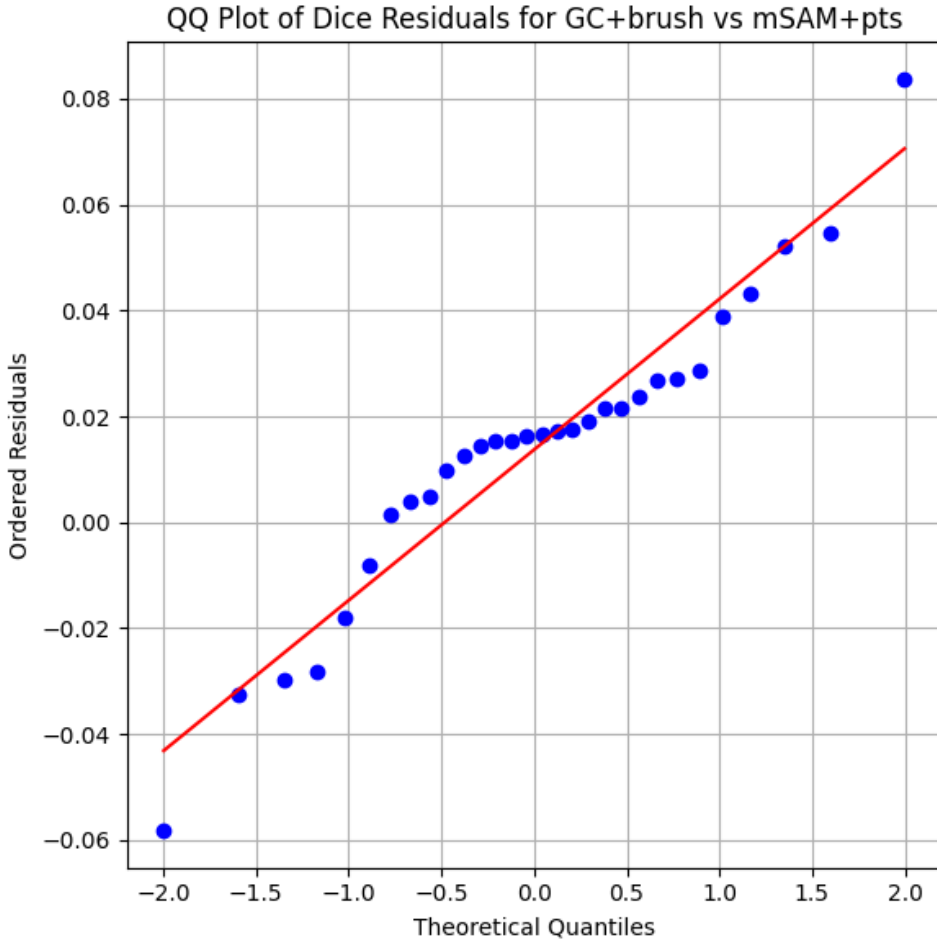


Figure 4.4: QQplot of the residuals of the paired t-test comparing GC+brush and mSAM+pts. The residuals appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test for normality of these residuals yielded a p-value of $0.647 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected.

4.1.4 Fairness of the experiment

4.1.4.1 Data

The dataset used for evaluation was the same as the one used for hyperparameter optimization of models, which could be seen as a potential source of bias. Additionally, the dataset of 30 images is quite small, which makes it more susceptible to noise and outliers, although sufficiently sized to support the statistical analysis. Moreover, the ground truth annotations of the dataset were heavily biased, even as they were manually created and registered to the raw images. Therefore, Dice scores obtained from the evaluation were unreasonably low, and the performance of the models is not properly reflected by these scores. However, for comparison between models, the relative differences in Dice scores were still meaningful, as all models were evaluated on the same imperfectly aligned dataset.

4.1.4.2 Model implementation

In the case of the fairness of the SAM implementation chosen, the experiment obviously only evaluates the performance of an untuned version. This implementation had most certainly never seen Linear A tablet images during training, as it was pretrained by default on the large and diverse dataset of natural images SA-1B. Furthermore, the model implementation used makes use of MobileSAMv2 with its small ViT image encoder TinyViT (**TinyViT**), which can compromise on segmentation quality. Lastly, SAM was restricted by the automatic nature of this experiment, as its intended use is interactive segmentation. Thus, it does not come as a surprise that mSAM underperforms in comparison to the classical baselines. The results therefore suggest a need for human intervention for the optimal placement of prompts to provide the best conditions for SAM, as it otherwise underperforms.

4.2 Interactive segmentation performance

...The results therefore suggest a need for a generalized and computationally efficient SAM implementation to be tuned to the greater domain of inscriptions, as the default model is not sufficient enough to prove its potential capabilities there. For this potential tuning to be possible, there is a need of a sufficiently sized one-class-labeled dataset of general inscription segmentations.

4.3 Workflow impact

CHAPTER 5

Conclusion

5.1 Summary of findings per research question

5.1.1 RQ1: Segmentation performance

5.1.2 RQ2: Degree of automation

5.1.3 RQ3: Workflow efficiency

5.2 Overall contribution

5.3 Future Work

5.3.1 Methodological extensions

5.3.2 Dataset expansion and diversification

5.3.3 Further automation opportunities

APPENDIX A

An Appendix

A.1 Non-interactive segmentation results

A.1.1 Hyperparameter optimization

Table A.1: Best TPE hyperparameter optimization results for CannyFill after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	22
$\sigma_{bilateral}$	21
$d_{closing}$	15
σ_{canny}	$9.42 \cdot 10^{-1}$

Table A.2: Best TPE hyperparameter optimization results for Gaussian after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	25
$\sigma_{bilateral}$	53
C	7
$d_{gaussian}$	31

Table A.3: Best TPE hyperparameter optimization results for mSAM+pts after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	5
σ	144
C	9
$d_{gaussian}$	11
n_{fgd_pts}	214
n_{bgd_pts}	1612
$d_{gap_erosion}$	11

Table A.4: Best TPE hyperparameter optimization results for GC+brush after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	25
$\sigma_{bilateral}$	31
C	10
$d_{gaussian}$	31
$d_{prb_erosion}$	5
$iters$	7

A.1.2 Raw evaluation data

Table A.5: Raw Dice score data on easy images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT10a	0.232	0.254	0.254	0.132	0.057	0.233
HT118	0.127	0.146	0.162	0.111	0.002	0.139
HT12	0.241	0.268	0.259	0.192	0.048	0.218
HT17	0.179	0.229	0.226	0.07	0.03	0.263
HT22	0.125	0.189	0.192	0.137	0.057	0.163
HT26a	0.227	0.265	0.264	0.118	0.001	0.242
HT7a	0.117	0.125	0.136	0.117	0.02	0.159
HT7b	0.172	0.162	0.178	0.075	0.026	0.15
KHWc2104	0.119	0.452	0.453	0.069	0.027	0.496
PH2	0.187	0.222	0.233	0.098	0.046	0.218

Table A.6: Raw Dice score data on medium images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT154B	0	0.154	0.154	0.072	0.032	0.122
HT16	0.169	0.168	0.172	0.081	0.003	0.183
HT2	0.107	0.109	0.118	0.096	0.021	0.103
HT4	0.205	0.236	0.237	0.173	0.061	0.22
HT40	0.133	0.153	0.164	0.088	0.029	0.126
HT6b	0.172	0.171	0.188	0.075	0.004	0.166
HT85b	0.115	0.105	0.108	0.062	0.035	0.096
HT8b	0.217	0.28	0.279	0.11	0.052	0.196
HT90	0.138	0.182	0.167	0.14	0.012	0.17
MA1a	0.172	0.253	0.247	0.144	0.09	0.25

Table A.7: Raw Dice score data on hard images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	GC+brush	Gaussian	Otsu	mSAM	mSAM+pts
HT1	0.098	0.125	0.114	0.043	0.049	0.09
HT3	0.079	0.086	0.083	0.058	0.046	0.067
HT30	0.092	0.1	0.101	0.054	0.006	0.097
HT33	0.038	0.047	0.052	0.016	0	0.035
HT52a	0.009	0.1	0.094	0.065	0.006	0.076
HT85a	0.107	0.103	0.093	0.056	0.006	0.128
HT8a	0.153	0.206	0.203	0.059	0.041	0.157
HT94b	0.07	0.074	0.078	0.039	0.017	0.06
HT9a	0.145	0.224	0.198	0.152	0.02	0.235
HT9b	0.071	0.118	0.116	0.071	0	0.077

A.1.3 Summary statistics

Table A.8: Summary Statistics per Model of Dice Score on all images.

Model	Mean Dice	Standard deviation	Standard error
CannyFill	0.134	0.062	0.011
GC+brush	0.177	0.083	0.015
Gaussian	0.177	0.082	0.015
Otsu	0.092	0.042	0.008
mSAM+pts-bilateral-filter-single	0.169	0.09	0.016

Table A.9: Summary Statistics per Model of Dice Score on easy images.

Model	Mean Dice	Standard deviation	Standard error
CannyFill	0.173	0.049	0.016
GC+brush	0.231	0.093	0.029
Gaussian	0.236	0.088	0.028
Otsu	0.112	0.037	0.012
mSAM+pts-bilateral-filter-single	0.233	0.103	0.033

Table A.10: Summary Statistics per Model of Dice Score on medium images.

Model	Mean Dice	Standard deviation	Standard error
CannyFill	0.143	0.062	0.02
GC+brush	0.181	0.058	0.018
Gaussian	0.183	0.055	0.018
Otsu	0.104	0.037	0.012
mSAM+pts-bilateral-filter-single	0.172	0.049	0.015

Table A.11: Summary Statistics per Model of Dice Score on hard images.

Model	Mean Dice	Standard deviation	Standard error
CannyFill	0.086	0.044	0.014
GC+brush	0.118	0.056	0.018
Gaussian	0.113	0.049	0.016
Otsu	0.061	0.035	0.011
mSAM+pts-bilateral-filter-single	0.102	0.057	0.018

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk